IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 20 August 2024, accepted 15 September 2024, date of publication 20 September 2024, date of current version 2 October 2024.

*Digital Object Identifier 10.1109/ACCESS.2024.3464611*

METHODS

# Denoising Method Based on Improved DBSCAN for Lidar Point Cloud

**ZHE ZHAO[1], WEILONG ZHOU[2], DENGHUI LIANG[2], JUNTAO LIU[2], AND XIAOBAO LEE[2]**
[1]College of Railway Transportation, Hunan University of Technology, Zhuzhou 412007, China
[2]College of Electrical and Information Engineering, Hunan University of Technology, Zhuzhou 412007, China

Corresponding author: Xiaobao Lee (lixiaobao200801@126.com)

**ABSTRACT** Lidar 3D point cloud data often contains significant noise, which affects the accuracy and reliability of subsequent data analysis. This study aims to propose an improved adaptive DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm to more effectively remove noise from LiDAR 3D point cloud data, ensuring high noise reduction accuracy and retention of original points, whereas maintaining good adaptability and robustness across different point cloud distributions and noise levels. The method determines the Eps range by fitting the inflection point of the distance matrix curve and identifies suitable parameters for point cloud data processing by finding the optimal Calinski-Harabasz index value. Experimental results show that this method achieves noise reduction accuracy and original point retention rates of over 90% when processing different types of point cloud data, whereas effectively preserving environmental and target information. This method addresses the limitations of adaptive parameter determination in existing DBSCAN-based denoising research by dynamically adjusting parameters according to the density characteristics of LiDAR point cloud data, meeting the needs of different point cloud distributions and noise levels.

**INDEX TERMS** Three-dimensional point cloud, noise, DBSCAN, adaptive.

## I. INTRODUCTION

Lidar emits laser beams and measures the flight time of the laser beams between the transmitter and the target surface to obtain distance information. Combined with lateral 2D coordinates, this results in the 3D positional information of the target, generating point cloud data. Lidar systems are commonly used in mapping, environmental perception, and target recognition. However, due to sensor errors, environmental interference, and other factors, point cloud data often contains outliers and stray points, which degrade the quality of subsequent data processing and analysis. Traditional denoising methods primarily include statistical filtering, bilateral filtering, and radius filtering based on various algorithmic libraries [1], [2], [3], [4], [5]. In recent years, besides traditional filtering methods, new filtering approaches have emerged. Cheng et al. [6] proposed projecting 3D point clouds onto a 2D plane based on grid PCA (Principal Component Analysis), then performing 3D reconstruction and denoising using a K-nearest neighbor-based segmentation method. Zhang et al. [7] introduced an unsupervised deep point cloud data denoising method guided by density priors, utilizing deep networks to obtain denoised point cloud data by calculating the probability of point cloud data lying on the true underlying surface.

Inspired by clustering in one-dimensional and two-dimensional data, the DBSCAN algorithm has gradually been applied to point cloud denoising. Clustering is widely used in fields such as image processing [8], spatial data analysis [9], anomaly detection [10], and neural networks [11]. DBSCAN identifies clusters based on local density rather than strict distance metrics. Its robustness to noise and flexibility in handling various cluster shapes make it particularly effective in dealing with clusters that have fuzzy and irregular boundaries. Ma et al. [12] proposed a DBSCAN algorithm combined with wave spectrum analysis to successfully eliminate noise photons below the water surface. Wei et al. [13] performed point cloud clustering and denoising

The associate editor coordinating the review of this manuscript and approving it for publication was Massimo Cafaro.

by calculating point density. Zhao et al. [14] divided point cloud data into voxel grids to reduce the search scope of the DBSCAN algorithm, while Tao et al. [15] used a gridding network to find the densest grid of point clouds to determine the clustering radius, completing the lidar point cloud filtering. Zhang et al. [16] proposed a DBSCAN model using an elliptical search area to improve ground height estimation accuracy. Subsequently, Popescu et al. [17] Use local statistics and clustering methods to identify background noise in photon-counting LiDAR data.

Traditional filtering methods typically rely on fixed processing rules. Although there are adaptive filtering techniques to address this problem [18], [19], they still tend to focus on local data processing. When dealing with nonlinear or irregular data structures, such methods may lead to excessive smoothing of point clouds, potentially resulting in the loss of important information. In contrast, clustering, as a machine learning approach, leverages the inherent structure of the data to preserve and extract key information, making it more effective in handling complex data patterns. However, these algorithms have not thoroughly considered the selection of the two critical parameters in the DBSCAN algorithm, Eps (Epsilon) and Minpts (Minimal points), mainly relying on manual selection or empirical settings. The parameter Eps is defined as the distance threshold between a point and neighboring points, whereas the parameter Minpts means the minimum number of points required in the neighborhood. The effectiveness of denoising varies with different parameter values. To address this problem, this paper proposes a new lidar point cloud denoising algorithm that optimizes the two DBSCAN parameters to achieve adaptive denoising.

## II. METHOD

### A. PRINCIPLE OF POINT CLOUD DATA DENOISING USING ADAPTIVE DBSCAN ALGORITHM

The main idea of using DBSCAN for denoising is to divide data points into high-density and low-density regions based on the density around the data points, and to perform clustering by analyzing the relationships between these density regions. Since noise points are usually located in low-density regions, whereas valid signal points are in high-density regions, the DBSCAN algorithm can effectively distinguish noise points from valid signal points.

In the DBSCAN algorithm, a core point is a data point that has at least 'Minpts' neighboring points within a given neighborhood, whereas a border point is a data point that has fewer than 'Minpts' neighboring points within its neighborhood but is reachable from a core point. Therefore, points that are neither core points nor border points are considered isolated points, which are density-reachable points [20]. Based on the density-based clustering concept, core points are identified in the point cloud. All points within the neighborhood radius of each core point are considered density-reachable points, forming part of a cluster. The cluster is gradually expanded by adding points that are adjacent and density-reachable from the core point until it can no longer be expanded, forming a complete cluster. Noise points, which typically do not have density-reachability, cannot form tight clusters with other data points and are thus identified as noise points that are not assigned to any cluster.

As shown in Fig.1, the red points represent core points, and the circles drawn around the core points represent the Eps range. Each circle contains at least 7 points, hence $Minpts = 7$. Core sample X is directly density-reachable from core sample P, core sample P is directly density-reachable from core sample Q, core sample Q is directly density-reachable from sample Y, and core sample X is directly density-reachable from sample M. Therefore, core sample X is density-reachable from sample Y, and sample M is density-connected to sample Y. Similarly, core sample X' is density-reachable from sample Y', and sample M' is density-connected to sample Y'. The blue arrows indicate direct density-reachability.

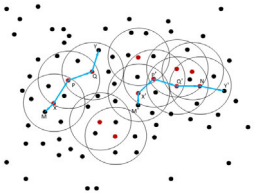

**FIGURE 1.** Basic concepts defined by DBSCAN (*Minpts* = 7).

However, Eps and Minpts are critical factors in determining the density of point clouds. Static parameter settings may lead to over-clustering or under-clustering. The adaptive DBSCAN algorithm incorporates local density information of the point cloud to dynamically adjust the values of Eps and Minpts. By utilizing the density reachability relationship with core objects, it incrementally clusters the point cloud signal points into clusters and marks points that cannot be density-reached to any core object as noise. This achieves noise reduction in point cloud data.

### B. STEPS OF THE ADAPTVE DBSCAN ALGORITHM

The denoising process of point cloud data based on the adaptive DBSCAN algorithm mainly includes the following key steps:

#### 1) CALCULATE CANDIDATE EPS AND MINPTS BASED ON THE POINT CLOUD SET

By analyzing the distribution characteristics of the point cloud set, calculate the appropriate neighborhood radius and the minimum number of points to be used as parameters for the DBSCAN algorithm. The pseudocode for the standard DBSCAN algorithm is shown in Algorithm 1.

In the DBSCAN algorithm, the K-dist curve reveals the density characteristics of data points by plotting the distance to the K-th nearest neighbor for each data point. Typically, the K-dist curve exhibits a clear ''elbow'' or ''knee,'' which signifies a transition from high-density to low-density regions.

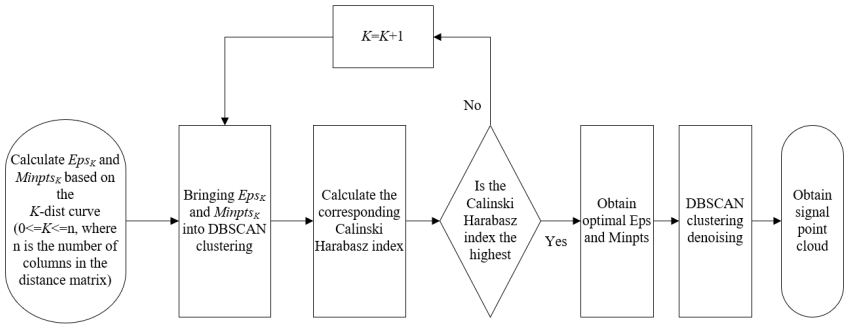


**FIGURE 2.** Steps of adaptive DBSCAN algorithm.

**Algorithm 1** DBSCAN

**Input:** *data, Minpts, Eps*

*class*←*0*
*D*←*pdist2(data, data)*
**for***each point*$p_i$ **in** *data* **do**
  **if** *class*$_{p_i}$ = *undefined* **then**
    *N*←*find(D(i,:)*≤*Eps)*
    **if** *numel (Neighbors)*< *Minpts*
      *isnoise (i)=true*
    **else**
      *C*←*C+1, IDX*$_i$ ← *C*
      *k*←*1*
      **While** *true*
        *j*=$N_k$
        **If** *j is not visited:*
          *visited(j)=true, N2*←*find (D (j,:)*≤*Eps)*
          **if** *numel(Neighbors2)*>= *Minpts*
            *Neighbors*← *[Neighbors Neighbors2];*
          **end if**
        **end if**
        **if** *IDX*$_j$ = = **0**
          *IDX*$_j$ ← *C*
        **end if**
        *k*←*k+1*
        **if** *k* > *numel(N)*
        **end if**
      **end while**
  **end if**
**end for**

In high-density areas, the distance to the K-th neighbor is relatively small, whereas in low-density areas, this distance increases significantly. The Y-axis value (the distance value) at the elbow is considered an effective choice for Eps because it marks the critical point of density change, allowing high density regions to be identified as clusters and low-density regions to be recognized as noise. Due to the potential presence of noise or irregularities in the actual data distribution, directly identifying the inflection point from the raw K-dist curve may not be sufficiently accurate. By performing polynomial fitting with an $R^2$ threshold on the K-dist curve, the data's fluctuations can be smoothed, making the curve's shape more representative of the overall data trend. This approach provides a smoother curve, facilitating the accurate identification of density change inflection points. The FindMaxCurvatureValues algorithm is displayed in Algorithm 2.

**Algorithm 2** FindMaxCurvatureValues

**Input:** *data*
**Output:** *y*

*n*←*size(data,1)*
*Dist*←*pdist2(data,data)*
*DEps*←sort(*Dist*)
*k*←1
$R^2$ ←0
**while** $R^2$<0.99 **and** *k* <70**do**
  *p*←*Polyfit(DEps(:,n),k), yfit*←*polyval(p, (1:n)');*
  *yresid*← *DEps(:, n) - yfit;*
  *SSresid*← *sum(yresid.^2);SStotal*←*(n-1)* ∗ *var(DEps(:,n));*
  $R^2$ = *1 - SSresid / SStotal*
  **if** $R^2$ < *0.99* **then**
    *k* ←*k+1*
  **else**
  **end if**
**end while**
*derivative1*← *polyder(p)*
Extract the maximum turning point *y*
**return** None

Regarding the selection of Minpts, in high-dimensional space, to ensure that there are enough points within the neighborhood to form a valid cluster, Minpts typically needs to be set to a higher value, often twice the dimensionality of the point cloud data. However, in practical applications, the point cloud density obtained from LIDAR is not uniformly distributed, so a fixed Minpts value may lead to significant errors. Choosing a Minpts value that is too small might fail to effectively delineate the cluster boundaries, whereas a value that is too large may result in too many points being classified as noise. Therefore, Minpts can be adjusted by calculating the mathematical expectation of the number of points within

the neighborhood for all data points. To ensure that each cluster's core points have sufficient neighboring points, the expectation value can be multiplied by a factor for Minpts adjustment. The CalculateMinPts algorithm is displayed in Algorithm 3.

**Algorithm 3** CalculateMinPts

**Input:** *Eps, data, beta*
**Output:** *Minpts*
1: $n \leftarrow size(data,1)$
2: **for** each $i$ **from** $1$ **to** $n$ **do**
3: $TotalNeighbors_i \leftarrow sum(pdist2(data(i,:), data) <= Eps)$
4: **end for**
5: $Minpts \leftarrow ceil(beta * sum(TotalNeighbors) / n)$

#### 2) CALCULATE OPTIMAL PARAMETERS USING THE CALINSKI-HARABASZ INDEX

Utilize the CH (Calinski-Harabasz) index to evaluate the performance of DBSCAN clustering under different parameters and select the parameters that maximize the index value as the optimal parameters.

The core of DBSCAN lies in defining clusters based on density, so the degree of point aggregation within clusters and the separation between different clusters are crucial for clustering performance. In parameter tuning, the CH index serves as an effective metric for guiding parameter selection. The CH index determines the within-cluster compactness by evaluating the average distance of points to the cluster center. A higher CH index value indicates greater aggregation within clusters and clearer separation between clusters. The withincluster variance represents the compactness of points within the same cluster, whereas the between-cluster variance reflects the separation between different clusters. By measuring the ratio of within-cluster variance to between-cluster variance, the CH index provides a comprehensive assessment of clustering quality. Maximizing the CH index helps find the optimal parameter combination for the data distribution and algorithm, thus optimizing clustering performance. Calculating the CH index for different parameter settings allows for selecting the parameter combination that best fits the data distribution, thereby improving clustering results.

#### 3) DBSCAN CLUSTERING AND DENOISING

Use the DBSCAN algorithm to cluster the data, separating noise points into individual clusters to achieve data denoising and effective clustering.

After obtaining the optimal parameter set, DBSCAN starts with a core point and first includes the core point and all points within its Eps neighborhood into a cluster. This means that all points within the Eps radius of the core point, including the core point itself, are considered part of the cluster. Once the core point and its neighborhood points are included in the cluster, the algorithm examines the neighborhoods of these points. If any of these points are also core points, their Eps neighborhoods are checked and added to the cluster as well. This expansion process continues until all density-reachable points are included in the cluster, meaning the cluster grows until no new density-reachable points can be found. In this manner, DBSCAN forms clusters that are composed of core points and their density-reachable points, creating clusters with a dense structure. Ultimately, points that meet the density requirements are assigned to a cluster, whereas those that cannot be connected to any core point by density are marked as noise points. The Adaptive DBSCAN algorithm is displayed in Algorithm 4.

**Algorithm 4** Adaptive DBSCAN

**Input:** *data, beta*
1: $best\_Eps \leftarrow 0$
2: $best\_Minpts \leftarrow 0$
3: $best\_CH \leftarrow -inf$
4: $n \leftarrow size(data,1)$
5: **for** each $i$ **from** $1$ **to** $n$ **do**
6: $Eps \leftarrow FindMaxCurvatureValues(data)$
7: $Minpts \leftarrow calculateMinPts(Eps, data, beta)$
8: $class \leftarrow DBSCAN(data, Minpts, Eps)$
9: $CH = calinski\_harabasz\_index(data, class)$
10: **if** $CH > best\_CH$ **then**
11: $best\_CH \leftarrow CH$
12: $best\_Eps \leftarrow Eps$
13: $best\_Minpts \leftarrow Minpts$
14: **else**
15: **end if**
16: **end for**
17: $class \leftarrow DBSCAN(data, best\_Minpts, best\_Eps)$

### C. CALCULATION OF CANDIDATE EPS AND MINPTS PARAMETERS

To better describe the parameter selection process, we take the Stanford Bunny point cloud model as an example for specific analysis. The original point cloud model consists of 10000 points, with an additional 2000 Gaussian noise points added to the original point cloud data, with a mean of 0 and a standard deviation of 0.1, forming a noisy point cloud as shown in Fig.5(a).

The determination of the Eps parameter range is based on the k-nearest neighbors' algorithm and the K-dist curve [21]. First, we use the distance matrix calculation method to measure the distances between points in the point cloud. Specifically, we represent the dataset as a matrix $\boldsymbol{D}_{\mathrm{n}*\mathrm{n}}$, where each row represents a data point and each column represents a feature. Then, employing the Euclidean distance as the distance metric, we compute the distances between each pair of data points in the dataset. The distance distribution matrix is represented as:

$$\boldsymbol{D}_{\mathrm{n}*\mathrm{n}} = \{d(i,j) | 1 \le i \le \mathrm{n}, 1 \le j \le \mathrm{n}\} \tag{1}$$

where $\boldsymbol{D}_{\mathrm{n}*\mathrm{n}}$ represents the distance matrix, where $d(i, j)$ denotes the distance between point cloud $i$ and point cloud $j$, and n is the total number of point clouds

Next, the distance matrix $\boldsymbol{D}_{\mathrm{n}*\mathrm{n}}$ is sorted in ascending order row-wise. The first column of the sorted matrix represents the distance of each point cloud object to itself, which is always 0. The elements in the K-th ($1 \leq \mathrm{K} \leq \mathrm{n}$ ) column of the sorted matrix are then sorted in ascending order. The number of point clouds, denoted as Index ( $1 \leq \mathrm{Index} \leq \mathrm{n}$ ), serves as the horizontal axis of the K-dist curve, whereas the sorted elements in each column represent the vertical axis. The Kdist curve illustrates the distance between the Index-th point and its nearest K neighbors.

Subsequently, the method of least squares is employed to fit each K-dist curve. The calculation method for the least squares is expressed as follows:

$$L = \sum_{k=1}^{\mathrm{n}} (y_k - f(x))^2 \tag{2}$$

The expression for the least squares polynomial fitting curve can be given as:

$$f(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_k x^k \tag{3}$$

where $k$ is the degree of the polynomial, and $\theta_i$ ($i = 1, 2, 3, \ldots, k$) are the coefficients of the polynomial terms. To determine the polynomial coefficients by solving a linear system: construct the matrix $\boldsymbol{X}$, where each row corresponds to a data point and the columns represent different polynomial terms. Therefore, the coefficient vector is:

$$\boldsymbol{\theta} = (\boldsymbol{X}^T \boldsymbol{X})^{-1} \boldsymbol{X}^T \boldsymbol{y} \tag{4}$$

$\boldsymbol{y}$ is the column vector of the actual observed values.

To find the most suitable k-degree polynomial curve fit, one needs to calculate the $R^2$ value, also known as the coefficient of determination, which is used to assess the goodness of fit of the model. It represents the proportion of the variance in the observed data that is explained by the model's predictions. The formula for $R^2$ can be expressed as:

$$R^2 = 1 - \frac{SS_{\mathrm{resid}}}{SS_{\mathrm{total}}} \tag{5}$$

where $SS_{\mathrm{resid}}$ is the sum of squared residuals, representing the sum of the squared differences between the observed values and the fitted values; $SS_{\mathrm{total}}$ is the total sum of squares, representing the sum of the squared differences between the observed values and their mean. The value of $R^2$ ranges between 0 and 1, where a value closer to 1 indicates a better fit of the model, explaining more variance in the observed data; conversely, a lower value indicates a poorer fit of the model. In this study, when $R^2 \geq 0.99$, the optimal $k$ is found. Figure 3 shows the original curve and the fitted curve when $K = 100$.

Next, the stationary points on the fitted K-dist curve are computed. In the K-dist curve graph, the shape of the curve typically changes significantly at a specific Eps value. By focusing on the changes in the curve, it is possible to more accurately identify the maximum stationary points in this region. The stationary points within this range usually correspond to the separation between noise points and actual clusters in the data. Choosing this range helps in better identifying noise points in the data and improves the effectiveness of the noise removal algorithm. According to the K-dist curve in Fig.3, the point where the curve starts to change significantly is near Index = 10000. Therefore, the points selected into the Eps parameter list are the maximum stationary points within the range of 0 to 10000, expressed as:

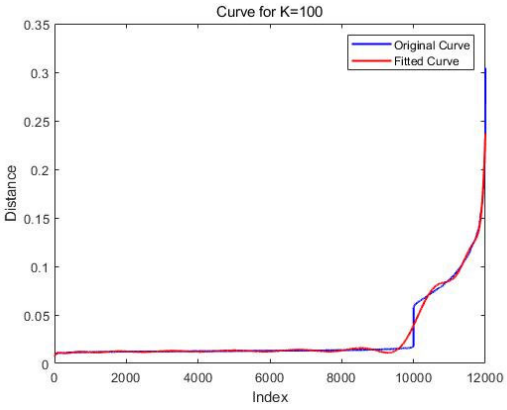


**FIGURE 3.** The original curve and the fitted curve when $K$=100.

$$x_{\mathrm{eps}}^n = \max\{x_1, x_2, \ldots\ldots, x_m\} \tag{6}$$

where $0 \leq m \leq 10000; 0 \leq n \leq 12000$; $x_1, x_2, \ldots\ldots, x_m$ are the points at which the first derivative of the polynomial $f(x)$ equals zero. The ordinate of the green dots in Figure 4 represents the calculated Eps when $K = 100$. From the figure, it can be observed that the algorithm accurately identifies the significant change points in the fitted curve, which reflect the boundary distance values between noise points and actual clusters when $K$ =100.

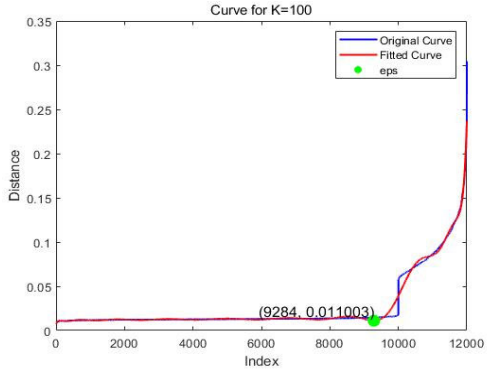


**FIGURE 4.** The Eps when $k$ =100.

Finally, calculate the number of point cloud objects contained in the neighborhood radius, and generate the

K-th Minpt*s*parameter using the mean value:

$$Minpts_K = \frac{\beta}{\mathrm{n}} \sum_{i=1}^{n} P_i \tag{7}$$

where $\beta$ is the noise suppression threshold, $0 \leq \beta < 1$ (in this study, $\beta$ is set to 0.5), n is the number of objects in the point cloud set $\boldsymbol{D}$, and $p_i$ is the number of objects in the Eps neighborhood of the i-th point cloud.

### D. OPTIMAL PARAMETER COMBINATION

The Calinski-Harabasz index [22], also known as the CH index, is a metric used to measure the effectiveness of the DBSCAN algorithm. It evaluates the compactness of clustering by computing the ratio of within-cluster dispersion to between-cluster dispersion. Its expression is given by:

$$CH = \frac{B}{W} \times \frac{\mathrm{n} - b}{b - 1} \tag{8}$$

where n is the total number of samples in the dataset; b is the number of clusters; $B$ is the between-cluster dispersion (between-cluster variance), defined as:

$$B = \sum_{i=1}^{s} n_i ||u_i - u||^2 \tag{9}$$

where $n_i$ represents the number of samples in the i-th cluster, $s$ is the number of clusters,$u_i$ represents the centroid of the i-th cluster, $u$ represents the global centroid, which is the mean of all samples, $||u_i - u||$ represents the euclidean distance between the centroid of cluster $i$ and the global centroid; W is the within-cluster dispersion (within cluster variance), defined as:

$$W = \sum_{i=1}^{s} \sum_{x_j \in C_i} ||x_j - u_i||^2 \tag{10}$$

where $C_i$ represents the sample set of the $i$-th cluster. $||x_j - u_i||$ represents the euclidean distance between each data point $x_j$ in the cluster $i$ and the centroid $u_i$ of that cluster.

The parameters set calculated in section C are used in DBSCAN, and the parameters that yield the highest CH index are considered the optimal parameters.

## III. DISCUSSION

### A. EXPERIMENTAL PARAMETERS AND RESULTS

We conducted simulations using MATLAB software. Firstly, we performed simulation analyses on the rabbit, pony, and elephant point cloud models from the Stanford database [23]. Gaussian noise with a mean of 0 and a standard deviation of 0.1 was added to the original point clouds to create noisy point clouds. Subsequently, to further validate the algorithm's performance in real and complex scenarios, we tested our algorithm using real residential area point cloud data publicly available from the National Science Foundation [24]. In the experiments, the point cloud data only included positional information. The optimal parameter sets calculated by our algorithm for the aforementioned point cloud data are listed in Table 1.

**TABLE 1.** The *Eps* and *Minpts* when $k = 100$.

| Type | *Eps* | *Minpts* |
|---|---|---|
| Rabbit | 0.0048375 | 7 |
| Pony | 0.0026562 | 7 |
| Elephant | 0.0209660 | 12 |
| Residential Area | 1.7636 | 45 |

After setting the above optimal parameters, the DBSCAN denoising effects for the rabbit, pony, elephant models and residential area point cloud are shown in Fig.5 to Fig.8, respectively. From the results displayed in Figures 5 to 8, it is evident that the algorithm proposed in this paper can accurately filter out noise whereas retaining clear target objects.

To observe the denoised point cloud more clearly, Cloud Compare was used to create magnified views of the front and side of the residential area, as shown in Fig.9. As can be seen from the figure, most of the noise points have been correctly filtered out, and environmental information such as trees has been retained.

### B. COMPARATIVE ANALYSIS

To verify the effectiveness of the algorithm, the Stanford Bunny point cloud data was used as an example and applied to radius filtering and the k-d tree (k-dimensional tree)-based statistical filtering algorithm. The denoising effects are shown in the Fig.10 and Fig.11.

To more intuitively demonstrate the filtering effects, unfiltered noise points are marked in red. In the 3D view of Fig.10(a), it can be clearly seen that radius filtering performs well in removing scattered noise, effectively eliminating most isolated noise points. However, from the planar view in Fig.10(b), it can be observed that whereas radius filtering removes noise, it also leads to significant data loss. Particularly in the main areas, some critical geometric information is mistakenly deleted during the filtering process, resulting in an incomplete point cloud model and the loss of certain details. In contrast, the point cloud data processed with statistical filtering better retains the main structure, avoiding the significant information loss seen with radius filtering. However, another problem arises: after filtering, whereas the main information is preserved, a large amount of residual noise remains, with these noise points scattered around the point cloud, affecting the overall quality. Although statistical filtering can maintain data integrity to a certain extent, its noise removal performance is inferior to that of radius filtering, especially when dealing with large areas of scattered noise, revealing certain limitations.

Compared with the proposed algorithm, these two methods still show obvious shortcomings in handling complex noise. The proposed improved algorithm dynamically adjusts parameters, not only effectively removing scattered noise but

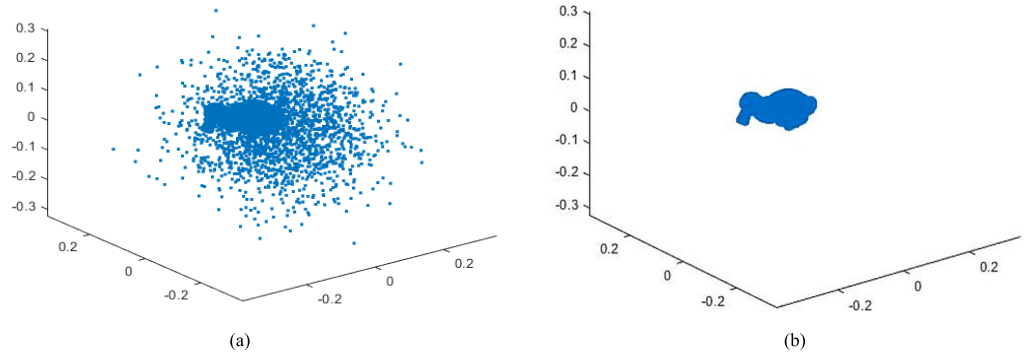


**FIGURE 5.** Denoising result of noisy rabbit point cloud data: (a) Before denoising, (b) After denoising.

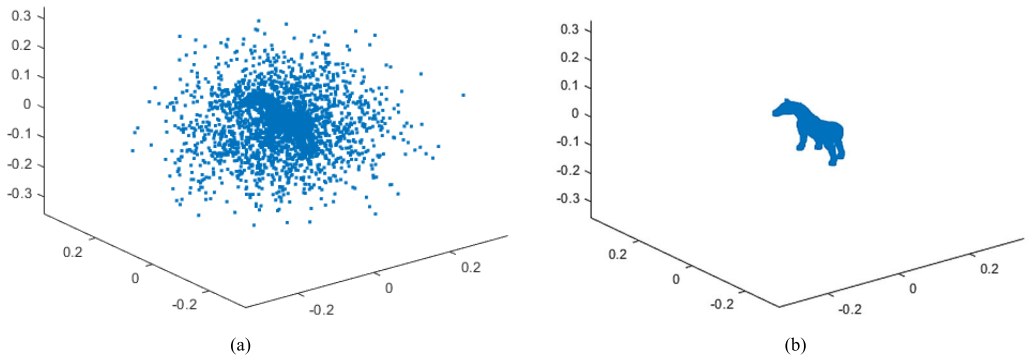


**FIGURE 6.** Denoising result of noisy pony point cloud data: (a) Before denoising, (b) After denoising.

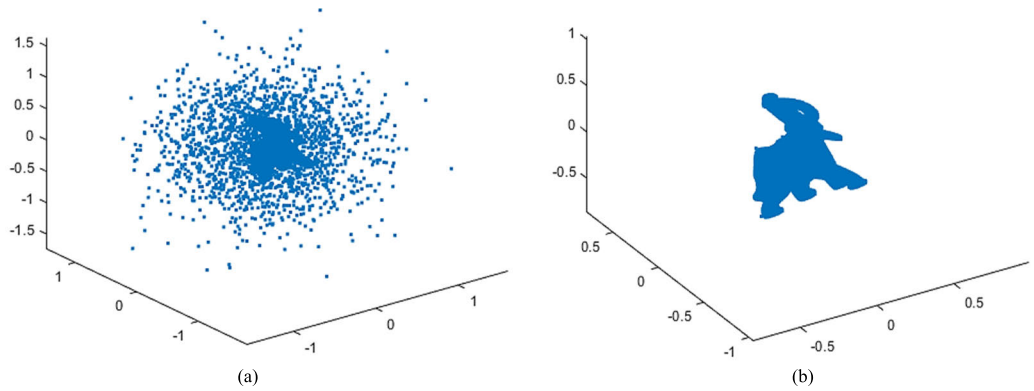


**FIGURE 7.** Denoising result of noisy elephant point cloud data: (a) Before denoising, (b) After denoising.

also better preserving the geometric structure of the point cloud data. It maintains the completeness of the main structure whereas reducing residual noise, significantly improving the quality of the point cloud. By conducting a deep analysis of the internal structure of the point cloud data, the new algorithm intelligently distinguishes between noise and valid data, avoiding the information loss caused by simple global filtering. This flexible adaptability allows the algorithm to exhibit greater robustness and accuracy when dealing with complex, irregular point cloud data, achieving efficient noise reduction without sacrificing detail. This also validates the algorithm's broad applicability and reliability across different scenarios and datasets.

### C. EVALUATION

Introduce metrics such as noise removal precision ($P_d$), noise recall rate ($R_d$), and origin preservation rate ($R_o$) to quantitatively compare the denoising effectiveness of filtering algorithms. Noise removal precision refers to the proportion of points correctly identified as noise by the algorithm among those actually noise, reflecting the accuracy of the

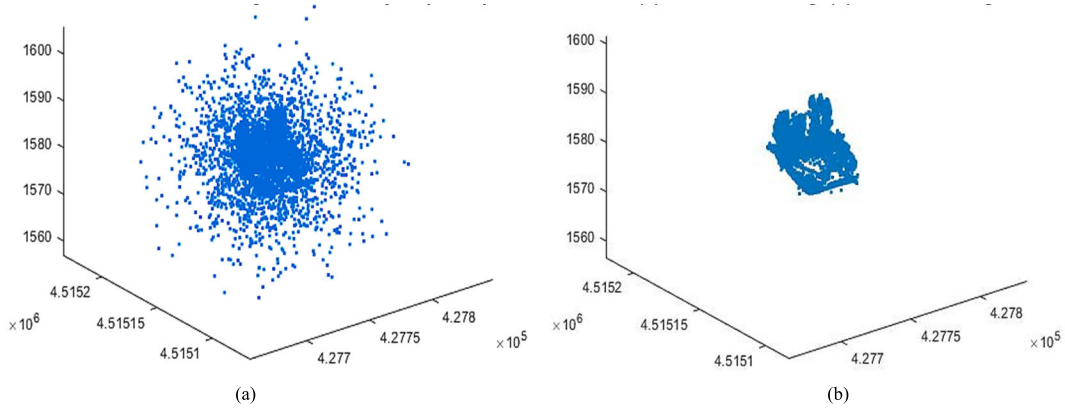


**FIGURE 8.** Denoising result of noisy residential point cloud data: (a) Before denoising, (b) After denoising.

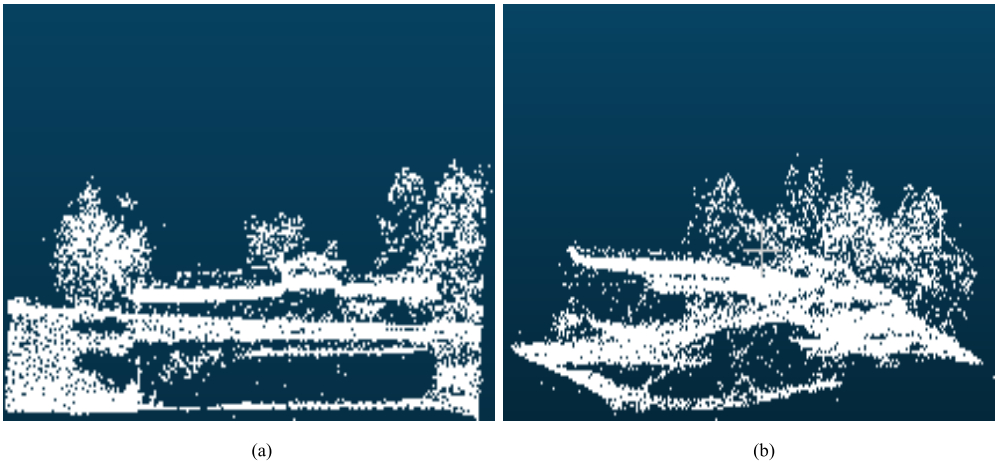


**FIGURE 9.** Denoised point cloud of residential area rendered by cloud compare: (a) Front View of Residential Area; (b) Side view of residential area.

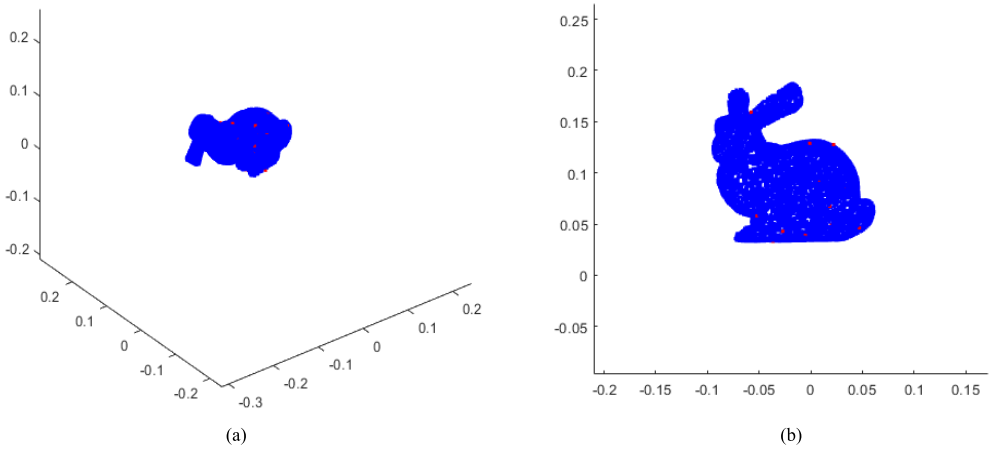


**FIGURE 10.** Radius filtering: (a) 3D view, (b) Planar view.

algorithm in identifying noise; noise recall rate refers to the proportion of true noise points correctly identified as noise by the algorithm among all true noise points, measuring the algorithm's ability to identify noise; origin preservation rate refers to the proportion of points in the original data not marked as noise that are correctly preserved in the

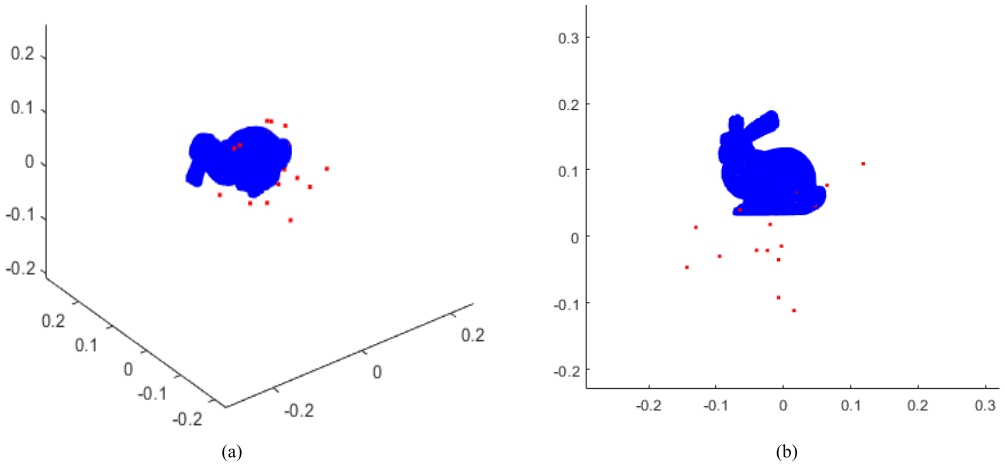


**FIGURE 11.** K-d tree -based statistical filtering: (a) 3D view, (b) Planar view.

clustering results, measuring the algorithm's ability to remove noise whereas preserving original information. By considering these metrics comprehensively, the performance of the algorithm can be more comprehensively evaluated. The formulas for each metric are as follows:

$$P_{\mathrm{d}} = \frac{N_{\mathrm{q}}}{N_{\mathrm{g}}} \tag{11}$$

$$R_{\mathrm{d}} = \frac{N_{\mathrm{q}}}{N_{\mathrm{y}}} \tag{12}$$

$$R_{\mathrm{o}} = \frac{N_{\mathrm{o}}}{N_{\mathrm{f}}} \tag{13}$$

In the formulas, $N_{\mathrm{q}}$ represents the number of filtered noise points, $N_{\mathrm{g}}$ represents the total number of noise points, $N_{\mathrm{y}}$represents the total number of filtered point clouds, $N_{\mathrm{o}}$ represents the number of signal points in the remaining point clouds, and $N_{\mathrm{f}}$represents the number of remaining points after filtering.

The quantities of $N$, $N_{\mathrm{q}}$, $N_{\mathrm{g}}$, $N_{\mathrm{y}}$, $N_{\mathrm{o}}$, $N_{\mathrm{f}}$ after denoising are listed in the table 2, where $N$ represents the total number of point clouds.

**TABLE 2.** Results after denoising.

| Type | $N$ | $N_{\mathrm{q}}$ | $N_{\mathrm{g}}$ | $N_{\mathrm{y}}$ | $N_{\mathrm{o}}$ | $N_{\mathrm{f}}$ |
|---|---|---|---|---|---|---|
| Rabbit | 12000 | 1964 | 2000 | 1971 | 9993 | 10029 |
| Pony | 22000 | 1986 | 2000 | 1999 | 19987 | 20001 |
| Elephant | 22000 | 1962 | 2000 | 1970 | 19992 | 20030 |
| Residential Area | 20184 | 1799 | 2000 | 2024 | 17959 | 18160 |

The evaluation metrics for the denoising algorithm proposed in this paper are presented in the following table 3.

For the Stanford bunny example, the values after denoising with the radius filtering and k-d tree-based Statistical Filtering algorithms are shown in the table below:

**TABLE 3.** Evaluation metrics after denoising.

| Type | $P_{\mathrm{d}}$ | $R_{\mathrm{d}}$ | $R_{\mathrm{o}}$ |
|---|---|---|---|
| Rabbit | 0.982 | 0.996 | 0.996 |
| Pony | 0.993 | 0.993 | 0.999 |
| Elephant | 0.981 | 0.996 | 0.998 |
| Residential Area | 0.900 | 0.889 | 0.989 |

**TABLE 4.** Denoising results of radius filtering and k-d tree based statistical filtering.

| Algorithm | $N_{\mathrm{q}}$ | $N_{\mathrm{y}}$ | $N_{\mathrm{o}}$ | $N_{\mathrm{f}}$ | $P_{\mathrm{d}}$ | $R_{\mathrm{d}}$ | $R_{\mathrm{o}}$ |
|---|---|---|---|---|---|---|---|
| Radius Filtering | 1977 | 2138 | 9839 | 9862 | 0.988 | 0.925 | 0.998 |
| K-d tree-based Statistical Filtering | 1978 | 2709 | 9269 | 9291 | 0.989 | 0.730 | 0.998 |

In the experimental process, a considerable amount of manual tuning and parameter optimization was performed to obtain the results shown in the table, which required substantial effort. Analysis of the data reveals that the $N_{\mathrm{y}}$ value is significantly high whereas the $N_{\mathrm{f}}$ value is notably low. This indicates that although the algorithm filtered out a substantial amount of noise, it incorrectly identified many signal points as noise, leading to a loss of effective points and a significant decrease in the noise recall rate.

As shown in the Table 4, the adaptive DBSCAN algorithm maintains high denoising precision, noise recall rate, and original point retention rate in point cloud models. However, due to the complexity of target objects and environmental information in residential scenes, noise can easily blend with the main body, resulting in a slight decrease in denoising precision and noise recall rate. Nevertheless, both the noise reduction accuracy and origin preservation rate can reach over

90%, whereas retaining effective environmental and target information.

## IV. CONCLUSION

This paper proposes a novel adaptive DBSCAN point cloud denoising method. Initially, potential Eps and Minpts parameters are calculated based on the k-nearest neighbors' algorithm and K-dist curve. These parameters are then input into the DBSCAN algorithm to solve for the maximum *CH*index, which corresponds to the optimal parameter set. Simulation experiments demonstrate that this algorithm is effective in clustering noise and signals in both point cloud models and complex scenes. The denoising precision, noise recall rate, and original point retention rate can all be maintained at above 90%. Moreover, it can effectively preserve target point clouds, addressing the shortcomings of manual parameter tuning and the inability to switch parameters based on point cloud transformations when using DBSCAN for denoising. This adaptability not only reduces the need for manual intervention but also enhances processing accuracy, providing a new direction for research in point cloud denoising.

However, future research needs to further reduce the algorithm's complexity to enable real time processing of large-scale point cloud data and validate its applicability in multi-view, large-scale scenes. The focus could also be on optimizing the algorithm design and utilizing deep learning and data-driven methods for more in-depth processing, to meet the precision requirements of various application scenarios and hardware conditions.

## APPENDIX

## NOMENCLATURE

| *Symbol* | *Meaning* |
|---|---|
| *Eps*(epsilon) | Neighborhood radius of a point. |
| *Minpts* (minimum points) | The minimum number of points required to form a dense region or cluster. |
| $x_{\text{eps}}^{n}$ | The Eps value for the *n*th point. |
| $Minpts_K$ | The Minpts corresponding to the K-th distance curve magnetic moment. |
| *CH* (Calinski-Harabasz Index) | Measures the separation between clusters and the cohesion within clusters. |
| $B$ | Trace of the between-cluster sum of squared deviations. |
| $W$ | Trace of the within-cluster sum of squared deviations. |
| $P_{\text{d}}$ | The measure of how effectively a noise removal process eliminates unwanted noise while preserving the original signal or data. |
| $R_{\text{d}}$ | The proportion of actual noise points that are correctly identified as noise |
| $R_{\text{o}}$ | Measures the extent to which the original data is preserved during the process of noise removal or other operations. |

## REFERENCES


[1] E. Leal, G. Sanchez-Torres, and J. W. Branch, "Sparse regularization-based approach for point cloud denoising and sharp features enhancement," *Sensors*, vol. 20, no. 11, p. 3206, Jun. 2020, doi: 10.3390/s20113206.

[2] F. Zhang, C. Zhang, H. Yang, and L. Zhao, "Point cloud denoising with principal component analysis and a novel bilateral filter," *Traitement du Signal*, vol. 36, no. 5, pp. 393–398, Nov. 2019, doi: 10.18280/ts.360503.

[3] N. Li, S. Yue, and B. Jiang, "Adaptive and feature-preserving bilateral filters for three-dimensional models," *Traitement du Signal*, vol. 37, no. 2, pp. 157–168, Apr. 2020, doi: 10.18280/ts.370202.

[4] B. Zou, H. Qiu, and Y. Lu, "Corrections to 'point cloud reduction and denoising based on optimized downsampling and bilateral filtering,'" *IEEE Access*, vol. 8, p. 159479, 2020, doi: 10.1109/ACCESS.2020.3020009.

[5] W. Guoqiang, Z. Hongxia, G. Zhiwei, S. Wei, and J. Dagong, "Bilateral filter denoising of LiDAR point cloud data in automatic driving scene," *Infr. Phys. Technol.*, vol. 131, Jun. 2023, Art. no. 104724, doi: 10.1016/j.infrared.2023.104724.

[6] D. Cheng, D. Zhao, J. Zhang, C. Wei, and D. Tian, "PCA-based denoising algorithm for outdoor LiDAR point cloud data," *Sensors*, vol. 21, no. 11, p. 3703, May 2021, doi: 10.3390/s21113703.

[7] J. Zhang, M. H. Nie, J. X. Wang, and X. P. Liu, "Unsupervised deep point cloud denoising algorithm guided by density prior," *J. Comput.-Aided Des. Comput. Graph.*, vol. 36, no. 2, pp. 283–293, 2024.

[8] E.-H. Kim, Z. Wang, H. Zong, Z. Jiang, Z. Fu, and W. Pedrycz, "Design of tobacco leaves classifier through fuzzy clustering-based neural networks with multiple histogram analyses of images," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 4698–4709, Mar. 2024, doi: 10.1109/TII.2023.3326533.

[9] B. Peng, G. Lin, J. Lei, T. Qin, X. Cao, and N. Ling, "Contrastive multi-view learning for 3D shape clustering," *IEEE Trans. Multimedia*, vol. 26, pp. 6262–6272, 2024, doi: 10.1109/TMM.2023.3347842.

[10] Y. H. Zhao, H. R. Li, X. Yu, N. Ma, T. Yang, and J. Zhou, "An independent central point OPTICS clustering algorithm for semi-supervised outlier detection of continuous glucose measurements," *Biomed. Signal Process. Control*, vol. 71, Jan. 2022, Art. no. 103196, doi: 10.1016/j.bspc.2021.103196.

[11] Z. Wang, E. H. Kim, S. KwunOh, W. Pedrycz, Z. Fu, and J. H. Yoon, "Reinforced fuzzy rule-based neural networks realized through streamlined feature selection strategy and fuzzy clustering with distance variation," *IEEE Trans. Fuzzy Syst.*, early access, Jul. 9, 2024, doi: 10.1109/TFUZZ.2024.3422414.

[12] Y. Ma, W. Zhang, J. Sun, G. Li, X. H. Wang, S. Li, and N. Xu, "Photon-counting LiDAR: An adaptive signal detection method for different land cover types in coastal areas," *Remote Sens.*, vol. 11, no. 4, p. 471, Feb. 2019, doi: 10.3390/rs11040471.

[13] S. Wei, N. X. Zhao, M. L. Li, and Y. H. Hu, "Single-photon denoising algorithm combining improved DBSCAN and statistical filtering," *Laser Technol.*, vol. 45, no. 5, pp. 601–606, 2021.

[14] K. Zhao, Y. C. Xu, Y. L. Li, and R. D. Wang, "Large-scale scattered point-cloud denoising based on VG-DBSCAN algorithm," *Acta Optica Sinica*, vol. 38, no. 10, pp. 370–375, 2018, doi: 10.3788/AOS201838.1028001.

[15] Z. Y. Tao, J. Q. Su, Z. Y. Dong, and X. P. Shan, "Research on three-dimensional laser radar point cloud filtering algorithm based on improved density clustering," *Appl. Laser*, vol. 43, no. 7, pp. 87–93, 2023.

[16] J. Zhang and J. Kerekes, "An adaptive density-based model for extracting surface returns from photon-counting laser altimeter data," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 4, pp. 726–730, Apr. 2015, doi: 10.1109/LGRS.2014.2360367.

[17] S. C. Popescu, T. Zhou, R. Nelson, A. Neuenschwander, R. Sheridan, L. Narine, and K. M. Walsh, "Photon counting LiDAR: An adaptive ground and canopy height retrieval algorithm for ICESat-2 data," *Remote Sens. Environ.*, vol. 208, pp. 154–170, Apr. 2018, doi: 10.1016/j.rse.2018.02.019.

[18] Y. Duan, C. Yang, and H. Li, "Low-complexity adaptive radius outlier removal filter based on PCA for LiDAR point cloud denoising," *Appl. Opt.*, vol. 60, no. 20, pp. E1–E7, 2021, doi: 10.1364/ao.416341.
[19] B. Liu and X. Li, "An adaptive dual-radius filtering algorithm based on LiDAR point cloud," *Acta Armamentarii*, vol. 44, no. 9, pp. 2768–2777, 2023, doi: 10.12382/bgxb.2022.1093.
[20] L. Duan, L. Xu, F. Guo, J. Lee, and B. Yan, "A local-density based spatial clustering algorithm with noise," *Inf. Syst.*, vol. 32, no. 7, pp. 978–986, Nov. 2007, doi: 10.1016/j.is.2006.10.006.
[21] L. Yin, H. Hu, K. Li, G. Zheng, Y. Qu, and H. Chen, "Improvement of DBSCAN algorithm based on K-Dist graph for adaptive determining parameters," *Electronics*, vol. 12, no. 15, p. 3213, Jul. 2023, doi: 10.3390/electronics12153213.
[22] S. P. Lima and M. D. Cruz, "A genetic algorithm using Calinski-Harabasz index for automatic clustering problem," *Revista Brasileira de Computação Aplicada*, vol. 12, no. 3, pp. 97–106, 2020, doi: 10.5335/rbca.v12i3.11117.
[23] L. Marc and T. Greg. *The Stanford 3D scanning Repository*. Accessed: Sep. 1, 2023. [Online]. Available: https://graphics.stanford.edu/data/3Dscanrep/
[24] A. H. Özcan and C. Ünsalan, *LiDAR Data Filtering and DTM Generation Using Empirical Mode Decomposition*. Accessed: Sep. 1, 2023. [Online]. Available: https://github.com/himmetozcan/EMD_Lidar

**ZHE ZHAO** was born in 2002. She received the bachelor's degree from Hanjiang Normal University. She is currently pursuing the master's degree in electronic information with Hunan University of Technology. Her main research interests include laser radar echo point cloud denoising. Her core paper has been accepted, and two patents are currently being applied.

**WEILONG ZHOU** was born in 1978. He received the master's degree. He is a Lecturer with the School of Electrical and Information Engineering, Hunan University of Technology. He has participated in two national science and technology fund projects, three provincial science and technology department projects, and led two provincial education department projects. In 2009, he guided students to participate in the National University Electronic Design Competition and won one second prize and two third prizes in the Hunan Province competition area. In 2012, he guided students to participate in the Hunan Province Electronic Design Competition for College Students and won one second prize and one winning prize. In 2013, he guided students to participate in the National University Electronic Design Competition and won one second prize in the country, one first prize in the Hunan Province competition area, one second prize, and one third prize. He also participated in the compilation of two textbooks and published over 20 academic papers, including eight EI-indexed articles. His main research interests include wireless sensor networks and embedded system design.

**DENGHUI LIANG** was born in 1998. He received the bachelor's degree in engineering from Longdong University, in June 2022. He is currently pursuing the master's degree in electrical engineering with Hunan University of Technology. His core paper has been accepted. His research direction is photovoltaic inverter microgrid integration.

**JUNTAO LIU** was born in 2000. He received the bachelor's degree in engineering from Hunan University of Technology, in 2022, where he is currently pursuing the master's degree in engineering. His main research direction is model predictive control of permanent magnet synchronous motors.

**XIAOBAO LEE** was born in February 1981. He received the bachelor's degree from Hunan University of Science and Technology, in 2006, the master's degree from Harbin Institute of Technology, in 2011, and the Ph.D. degree from the School of Astronautics, Harbin Institute of Technology, in 2017. He is currently with Hunan University of Technology, as a Graduate Supervisor, mainly engaged in the research, in fields of microgrid inverter control, optoelectronic information acquisition, and space optical communication technology. He has published four articles as the first author or corresponding author in foreign SCI journals, such as *Optics Express* and *Applied Optics*, and led two provincial and ministerial level scientific research projects.